cube-v1 · cube-v2 코드 분석과 구조적 차이

대상: Autocomputergraphics2026 저장소의 `cube-v1/`, `cube-v2/` · 환경: Qt 6, OpenGL 3.3 Core, CMake · 작성일: 2026-10-30

요약

- **GeometryEngine, 셰이더, 리소스, 투영 행렬, 빌드 설정은 두 버전이 같습니다.** 차이는 모두 `MainWidget` (`mainwidget.h` , `mainwidget.cpp`)에 있습니다.
- **두 버전 모두 카메라는 고정되어 있고, 마우스로 움직이는 것은 물체(주사위)입니다.** v2도 카메라를 움직이지 않습니다.
- v1 장면(물체 위치, 카메라 위치)이 `paintGL` 안에 숫자로 고정되어 있습니다. 마우스 회전은 "카메라가 정면을 보고 회전하지 않는다"는 숨은 가정 위에서만 맞습니다.
- v2 물체 자세와 카메라를 **멤버 변수(장면 상태)**로 분리하고, 뷰 행렬을 `lookAt` 으로 만들며, 마우스 회전축을 **화면 → 월드 → 물체 좌표계로 명시적으로 변환**합니다. 그래서 카메라가 어디에 있든 회전이 올바르게 동작합니다.
- 현재 설정(카메라가 (0,0,5)에서 원점을 봄)에서는 두 버전의 회전 계산 결과가 수학적으로 같습니다. 눈에 보이는 차이는 **처음 보이는 면**뿐입니다(v2는 시작할 때 y축 기준 −90° 회전).
- **2장 그림 1~5**에 월드 좌표계에서 카메라와 큐브의 위치, 카메라가 보는 범위, 마우스로 끌 때의 회전 방식을 정리했습니다. 7장 **그림 6**은 카메라를 옮겼을 때 v1과 v2의 차이를 보여 줍니다.

1. 공통 뼈대 (v1, v2 동일)

1.1 파일별 역할

파일	역할
`main.cpp`	OpenGL 3.3 Core 컨텍스트와 24bit 깊이 버퍼를 요청합니다 (`QSurfaceFormat::setDefaultFormat` , `QApplication` 생성 **전에** 호출). 그다음 `MainWidget` 을 띄웁니다.
`MainWidget`	장면 관리, 렌더링, 입력 처리를 모두 맡습니다. `QOpenGLWidget` 과 `QOpenGLFunctions_3_3_Core` 를 상속합니다.
`GeometryEngine`	큐브의 정점과 인덱스를 GPU 버퍼(VAO, VBO, IBO)에 올리고 그리는 일만 합니다.
`vshader.glsl`	`gl_Position = mvp_matrix * a_position` 을 계산하고, 텍스처 좌표(UV)를 프래그먼트 셰이더로 넘깁니다.
`fshader.glsl`	`fragColor = texture(cube_texture, v_texcoord)` 로 텍스처 색을 출력합니다.
`shaders.qrc` , `textures.qrc`	셰이더와 `cube.png` 를 실행 파일 안에 넣습니다. 코드에서는 `:/vshader.glsl` , `:/cube.png` 처럼 접근합니다.
`CMakeLists.txt`	`qt_add_executable` , `qt_add_resources` 로 빌드합니다. 두 버전의 차이는 빌드 대상 이름 (`cube-v1` , `cube-v2`)뿐입니다.

1.2 실행 흐름 (Qt가 함수를 부르는 순서)

프로그램 시작	`main()` → 컨텍스트 형식 설정 → `MainWidget.show()`. 초기 창 크기는 `sizeHint()`가 돌려주는 512×512
`initializeGL()` (1회)	OpenGL 함수 준비(실패 시 `qFatal`) → 실제 받은 OpenGL 버전 출력 → 배경색 검정 → `initShaders()` → `initTextures()` → `new GeometryEngine` → v2 `initScene()`
`resizeGL(w, h)` (창 크기 변경 시)	투영 행렬 P 계산: 시야각 45°, 종횡비 w/h, near 3, far 7
`paintGL()` (다시 그릴 때마다)	색, 깊이 버퍼 지우기 → 깊이 테스트, 뒷면 제거 켜기 → 텍스처와 셰이더 바인딩 → MVP 계산 후 uniform 전달 → `drawCubeGeometry()`
마우스 드래그	`mousePressEvent`에서 위치 저장 → `mouseMoveEvent`에서 회전 상태 갱신 → `update()` → `paintGL()` 다시 호출

1.3 GPU 데이터 구조 (`geometryengine.cpp`)

- **정점 24개**: 큐브 꼭짓점은 8개지만 면마다 텍스처 좌표가 달라서 면당 4개씩 따로 둡니다. 정점 하나는 `{QVector3D 위치, QVector2D UV}` = 20바이트입니다.
- **인덱스 36개**: 6면 × 삼각형 2개 × 3. 면마다 `(0,1,2)`, `(2,1,3)` 패턴입니다.
- 그리기: VAO 바인딩 → VBO, IBO 바인딩 → 정점 속성 연결(`a_position`: float 3개, 오프셋 0 / `a_texcoord`: float 2개, 오프셋 12) → `glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_SHORT)`
- 버텍스 셰이더의 `a_position`은 `vec4`인데 버퍼에서는 float 3개만 넘깁니다. 빠진 w 성분은 OpenGL 규칙에 따라 1.0이 됩니다.

1.4 좌표 변환 순서

물체 좌표 큐브 정점 (−1~1) → M (model) → **월드 좌표** 장면 전체 기준 → V (view) → **카메라 좌표** 카메라가 원점, −z를 봄 → P (projection) → **클립 좌표** 원근 투영 후

셰이더에는 `MVP = P · V · M`을 uniform `mvp_matrix`로 넘깁니다. 카메라가 월드에 놓인 자세를 C라고 하면 $V = C^{-1}$입니다.

2. 그림으로 보는 장면 배치와 회전

아래 그림의 좌표와 방향은 모두 v2 코드의 실제 값입니다. 큐브는 각 축 −1~1, 카메라는 `(0,0,5)`에서 `(0,0,0)`을 보고 위쪽 방향은 `(0,1,0)`, 투영은 시야각 45°, near 3, far 7입니다. 회전축과 회전 방향은 Qt의 `QMatrix4x4`, `QQuaternion`으로 직접 계산해서 확인했습니다.

2.1 월드 좌표계에서 카메라와 큐브의 위치

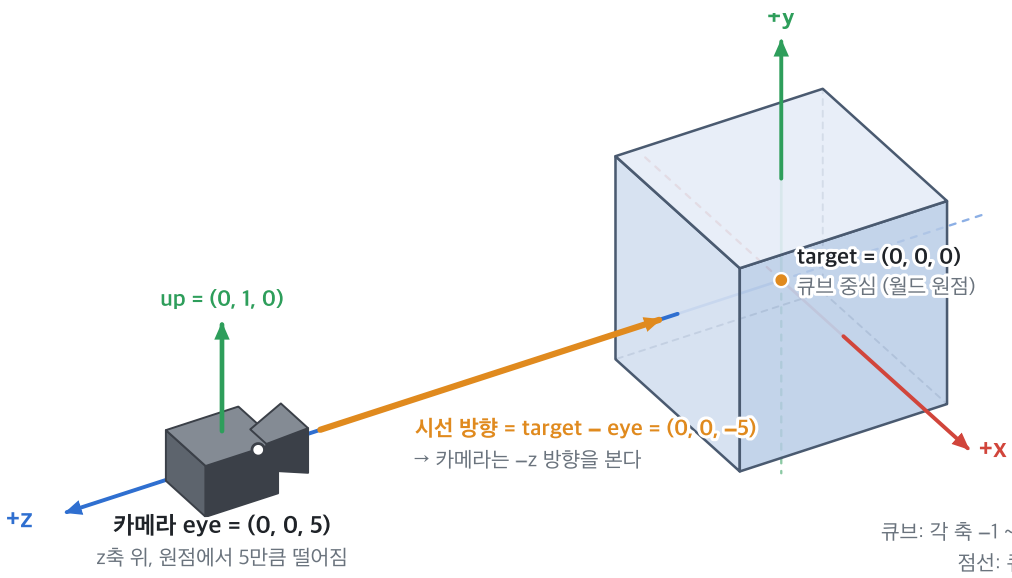


그림 1. 큐브의 중심은 월드 원점에 있고, 카메라는 +z축 위 `(0,0,5)`에 있습니다. 카메라는 원점(target)을 바라보므로 시선 방향은 −z이고, 카메라의 위쪽은 월드 +y입니다. 이 배치는 v1과 v2가 같습니다.

v2의 뷰 행렬 `lookAt(eye, target, up)`은 이 그림의 세 값을 그대로 넣은 것입니다. 카메라가 회전하지 않은 상태라서 **카메라의 x, y, z축이 월드의 x, y, z축과 같은 방향**입니다. v1의 회전 계산은 바로 이 조건 위에서만 맞습니다.

2.2 처음 보이는 면: v1과 v2의 차이

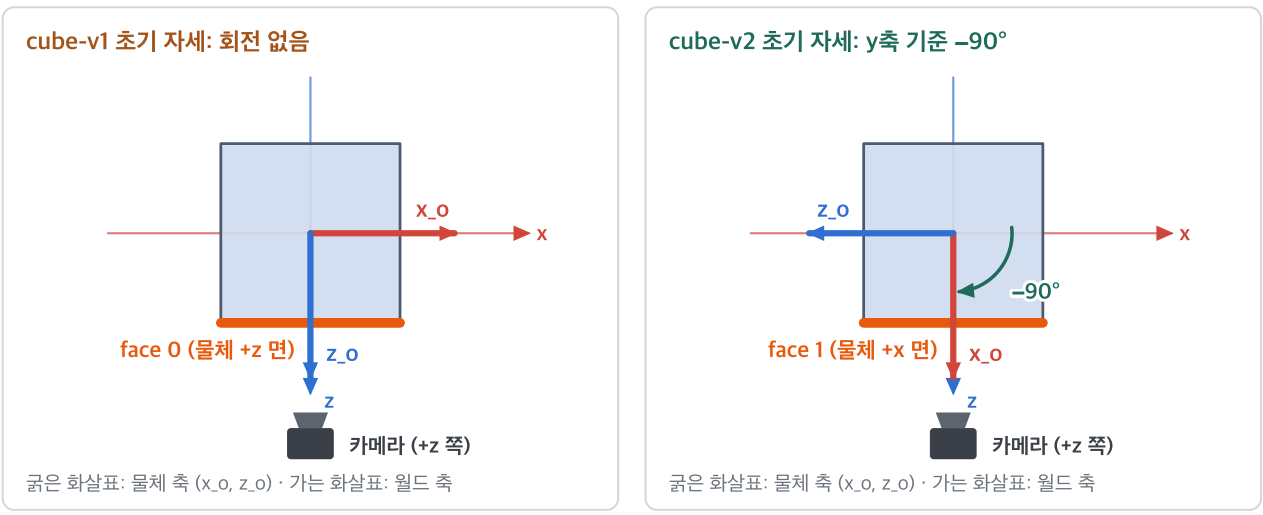
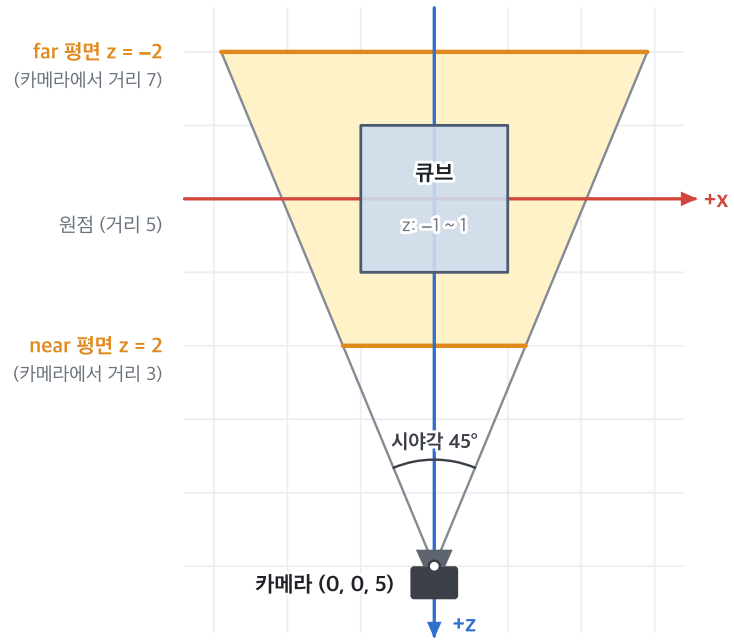


그림 2. v2는 `initScene()`에서 큐브를 y축 기준 −90° 돌려 둡니다. 그래서 물체의 x_o축이 월드 +z(카메라 쪽)를, z_o축이 월드 −x를 향하고, 카메라에는 face 0 대신 **face 1(물체 +x 면)**이 보입니다.

2.3 카메라가 바라보는 범위 (위에서 본 그림)



위에서 내려다본 그림 (+y가 종이 밖을 향함)

노란 영역 = 화면에 보이는 공간
near와 far 사이, 시야각 안쪽만 그려진다

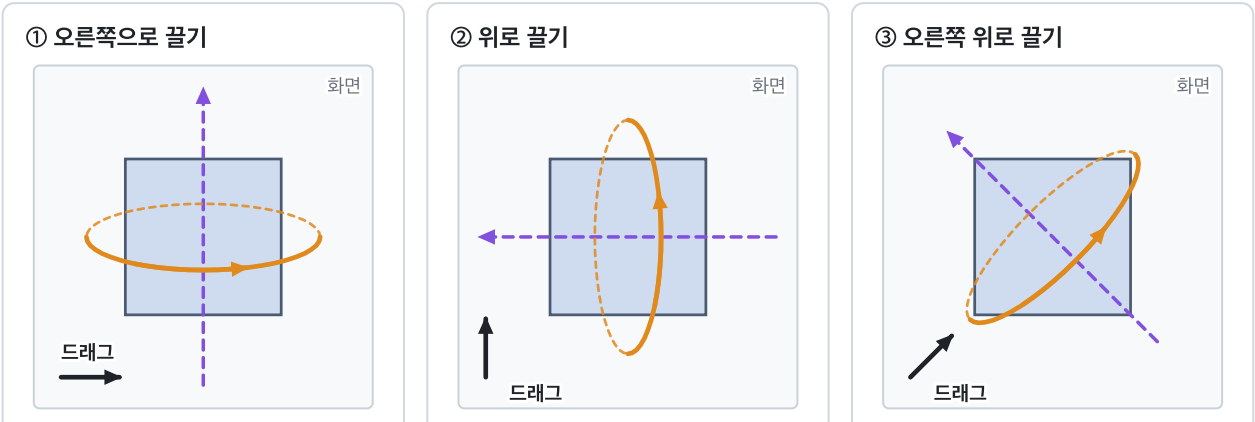
큐브는 카메라에서 거리 4 ~ 6
→ near 3 ~ far 7 범위 안이라 잘리지 않음

카메라를 멀리 / 가까이 옮기면
큐브가 이 범위를 벗어나 잘려 보인다

창이 정사각형(512×512)일 때 기준.
가로 시야각은 창 비율에 따라 달라진다

그림 3. 시야각 45°의 사각뿔 중에서 near 평면(거리 3)과 far 평면(거리 7) 사이만 화면에 그려집니다. 큐브는 카메라에서 거리 4~6에 있어서 이 범위 안에 들어갑니다.

2.4 마우스로 끌 때 회전이 이루어지는 방식



① 오른쪽으로 끌기
n_view = (0, 1, 0)
→ 세로축을 중심으로 회전
→ 앞면이 오른쪽으로 돈다
회전각 = 0.5° × 이동 픽셀 수

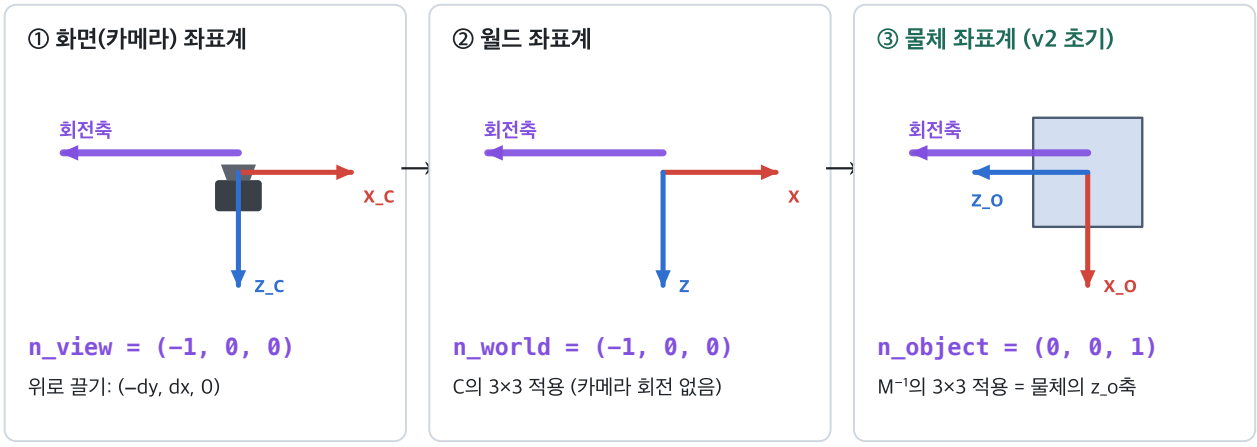
② 위로 끌기
n_view = (–1, 0, 0)
→ 가로축(왼쪽 방향)을 중심으로 회전
→ 앞면이 위로 돈다
회전각 = 0.5° × 이동 픽셀 수

③ 오른쪽 위로 끌기
n_view = (–1, 1, 0)/√2
→ 대각선 축을 중심으로 회전
→ 앞면이 오른쪽 위로 돈다
회전각 = 0.5° × 이동 픽셀 수

그림 4. 마우스를 끈 방향에 **수직인 화면 위의 축**(보라색 점선)을 중심으로 큐브가 돕니다. 주황색 실선은 큐브 앞쪽(카메라 쪽)이 움직이는 경로이고, 점선은 뒤쪽 경로입니다. 앞면은 항상 끈 방향으로 움직입니다.

1. 마우스 이동량 `diff`를 OpenGL 방향으로 바꿉니다. 화면 y는 아래로 증가하므로 `dy = -diff.y`입니다.
2. 회전축 `n_view = (-dy, dx, 0)`을 구합니다. 화면 평면 안에서 끈 방향을 90° 돌린 벡터입니다.
3. 회전각은 `|diff| × 0.5°`입니다. 끈 거리에 비례합니다.
4. 오른손 법칙: 축 방향으로 엄지를 두면 나머지 손가락이 감기는 방향으로 돕니다. 예를 들어 축 (0,1,0)으로 30° 돌리면 앞면 중심 (0,0,1)은 (0.5, 0, 0.87)로, 즉 오른쪽으로 이동합니다.

2.5 v2에서 회전축이 좌표계를 거치는 과정 (위로 끌기 예시)



보라색 화살표 = 같은 회전축(월드 −x 방향)을 좌표계마다 다른 숫자로 쓴 것. 위에서 내려다본 그림(y축은 종이 밖)

그림 5. 같은 회전축(월드 −x 방향)을 화면, 월드, 물체 좌표계에서 각각 숫자로 쓴 것입니다. 물체가 −90° 돌아가 있어서 물체 좌표계에서는 이 축이 `(0, 0, 1)`, 즉 물체의 z_o축이 됩니다.

v2는 이 축으로 `objectOrientation * R((0,0,1), θ)`를 계산합니다. 5.3절의 식에 따라 이 값은 `R((−1,0,0), θ) * objectOrientation`과 같습니다. 즉 월드 −x축을 중심으로 회전한 것과 같아서, 그림 4의 ②처럼 앞면이 위로 움직입니다.

3. cube-v1 분석: 장면이 그리는 코드 안에 들어 있는 구조

3.1 상태 (멤버 변수)

cube-v1/mainwidget.h:46–49

```
QMatrix4x4 projection;
QVector2D  lastMousePosition;
QQuaternion rotation;   // 물체 회전 하나뿐 (기본값: 회전 없음)
```

장면 상태는 **물체 회전값 하나**입니다. 물체 위치와 카메라는 변수로 존재하지 않습니다.

3.2 paintGL: 장면을 매번 그 자리에서 정의

cube-v1/mainwidget.cpp:169–185

```
QMatrix4x4 modelmatrix;                    // 지역 변수
modelmatrix.translate(0.0, 0.0, 0.0);      // Mo = To
modelmatrix.rotate(rotation);              // Mo = To x Ro

QMatrix4x4 cameraposmatrix;                // 지역 변수: 카메라가 월드에 놓인 자세 C
cameraposmatrix.translate(0.0, 0.0, 5.0);  // 카메라는 (0,0,5). 이동만 가능
QMatrix4x4 viewmatrix = cameraposmatrix.inverted();   // V = C^-1

QMatrix4x4 modelviewmatrix = viewmatrix * modelmatrix;
program.setUniformValue("mvp_matrix", projection * modelviewmatrix);
```

- 물체 위치 (0,0,0)과 카메라 위치 (0,0,5)가 **코드에 숫자로 박혀 있습니다.**
- 카메라는 이동만 할 수 있고 방향은 항상 −z(원점 쪽)를 봅니다.
- 행렬은 모두 지역 변수라서 `paintGL`이 끝나면 사라집니다.

3.3 mouseMoveEvent: 화면 축을 그대로 회전축으로 사용

cube-v1/mainwidget.cpp:38-65

```
QVector2D diff = pos - lastMousePosition;      // 직전 위치에서의 변화량
const double dx = diff.x();
const double dy = -diff.y();                   // 화면 y(아래로 증가) → OpenGL y(위로 증가)
QVector3D n_view = QVector3D(-dy, dx, 0.0).normalized();   // 이동 방향에 수직인 축
float angle = diff.length() * 0.5f;            // 1픽셀당 0.5°

rotation = QQuaternion::fromAxisAndAngle(n_view, angle) * rotation;   // 앞에 곱함
```

- 기존 회전의 **앞에** 곱하면 **월드 좌표계 축**을 중심으로 회전합니다.
- 그런데 넣는 축은 **화면(카메라) 좌표계 축** `n_view`입니다.
- 이 계산이 맞으려면 화면 축과 월드 축이 같아야 합니다. 즉 **카메라가 회전하지 않아야 합니다.** 이 가정은 코드 어디에도 드러나 있지 않습니다.

> **v1의 특징**: 마우스 처리 코드는 카메라와 물체의 행렬을 전혀 모릅니다. 장면 상태는 회전값 하나이고, 나머지는 `paintGL`에 고정된 값입니다. 처음 화면에는 +z 면(face 0)이 정면에 보입니다.

4. cube-v2 분석: 장면 상태를 변수로 두고, 행렬은 상태에서 계산하는 구조

4.1 상태 (멤버 변수)

cube-v2/mainwidget.h:48-63

```
// paintGL()에서 계산하고 mouseMoveEvent()에서 사용
QMatrix4x4 modelmatrix;       // 물체 좌표 → 월드 좌표 (M)
QMatrix4x4 cameraposmatrix;   // 카메라 좌표 → 월드 좌표 (C = V^-1)

QVector2D lastMousePosition;

// 물체 자세 (월드 기준), initScene()에서 초기화
QVector3D   objectPosition;
QQuaternion objectOrientation;

// 카메라 (월드 기준 위치, 바라보는 점, 위쪽 방향), initScene()에서 초기화
QVector3D cameraPosition;
QVector3D cameraTarget;
QVector3D cameraUp;

void initScene();
```

멤버 변수가 두 종류로 나뉩니다. **장면 상태**(물체 자세, 카메라)는 원래 값이고, **파생 행렬**(`modelmatrix`, `cameraposmatrix`)은 상태에서 계산된 결과를 입력 처리와 공유하려고 저장해 둔 것입니다.

4.2 initScene: 장면 초기값 설정

cube-v2/mainwidget.cpp:103-113 (initializeGL 마지막 줄 99에서 호출)

```
objectPosition    = QVector3D(0, 0, 0);
objectOrientation = QQuaternion::fromAxisAndAngle(0, 1, 0, -90);  // y축 기준 -90°

cameraPosition = QVector3D(0, 0, 5);   // 카메라 위치
cameraTarget   = QVector3D(0, 0, 0);   // 바라보는 점
cameraUp       = QVector3D(0, 1, 0);   // 위쪽 방향
```

y축 기준 −90° 회전은 +x 방향을 +z 방향으로 옮깁니다. 그래서 v2는 처음에 **+x 면(face 1)이 카메라 쪽**에 보입니다. v1과 눈에 보이는 차이는 이것뿐입니다.

4.3 paintGL: 장면 상태를 행렬로 바꾸기만 함

cube-v2/mainwidget.cpp:195-210

```
modelmatrix.setToIdentity();
modelmatrix.translate(objectPosition);     // Mo = To
modelmatrix.rotate(objectOrientation);     // Mo = To x Ro

QMatrix4x4 viewmatrix;
viewmatrix.lookAt(cameraPosition, cameraTarget, cameraUp);   // gluLookAt / glm::lookAt과 같음
cameraposmatrix = viewmatrix.inverted();   // C = V^-1 (멤버에 저장)

QMatrix4x4 modelviewmatrix = viewmatrix * modelmatrix;
program.setUniformValue("mvp_matrix", projection * modelviewmatrix);
```

현재 설정값에서 `lookAt((0,0,5), (0,0,0), (0,1,0))`은 v1의 `translate(0,0,5).inverted()`와 **같은 행렬**을 만듭니다.

4.4 mouseMoveEvent: 회전축의 좌표계 변환

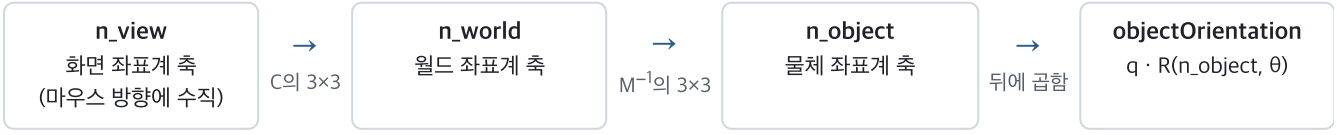
cube-v2/mainwidget.cpp:58-76

```
QVector3D n_view = QVector3D(-dy, dx, 0.0).normalized();   // v1과 같음: 화면 좌표계 축

// 화면 → 월드: 카메라의 회전(3x3)을 적용
QVector3D n_world  = cameraposmatrix.mapVector(n_view);
// 월드 → 물체: 물체 회전의 역(3x3)을 적용
QVector3D n_object = modelmatrix.inverted().mapVector(n_world).normalized();

// 물체 자신의 축으로 회전: 뒤에 곱함
objectOrientation = objectOrientation * QQuaternion::fromAxisAndAngle(n_object, angle);
```

- `mapVector()`는 4×4 행렬에서 이동 성분을 빼고 왼쪽 위 3×3 회전 부분만 적용합니다. 회전축은 위치가 아니라 방향이므로 이렇게 하는 것이 맞습니다.
- 기존 회전의 **뒤에** 곱하면 **물체 좌표계 축**을 중심으로 회전합니다. 그래서 축을 물체 좌표계로 변환해서 넣습니다.



5. 구조적 차이

5.1 데이터 흐름 비교

v1: 입력 처리와 렌더링이 서로 독립

mouseMoveEvent: n_view로 `rotation` 갱신 (앞에 곱함)

↓ update()

paintGL: 고정된 숫자로 M, C를 **지역 변수**로 만들고 MVP 계산

↓

행렬은 그리기가 끝나면 사라짐. 입력 처리는 카메라를 모름

v2: 상태 → 행렬 → 입력 → 상태로 도는 구조

장면 상태: 물체 자세 + 카메라 (initScene에서 초기화)

↓ paintGL

M, C를 계산해 **멤버에 저장**하고 MVP로 그리기

↓ mouseMoveEvent가 M, C를 읽음

축을 화면 → 월드 → 물체로 변환해 `objectOrientation` 갱신 → update()

5.2 비교표

관점	cube-v1	cube-v2
장면 정의 위치	`paintGL` 안에 숫자로 고정	멤버 변수(장면 상태) + `initScene()`
paintGL이 하는 일	장면 정의 + 행렬 계산 + 그리기	장면 상태를 행렬로 바꾸고 그리기만
물체 표현	회전 `rotation` 하나	위치 `objectPosition` + 방향 `objectOrientation`
카메라 표현	이동 행렬 하나. 방향 고정(−z)	위치, 바라보는 점, 위쪽 방향 → `lookAt`. 방향을 자유롭게 정할 수 있음
뷰 행렬 계산	`translate(0,0,5)` 의 역행렬	`lookAt(eye, target, up)`
행렬 수명	지역 변수 (그리는 동안만 존재)	M, C를 멤버로 저장해 입력 처리와 공유
입력 처리와 렌더링의 관계	독립	입력 처리가 직전 프레임의 M, C를 읽음
회전축 좌표계	화면 축을 월드 축으로 그냥 사용 (숨은 가정)	화면 → 월드 → 물체로 명시적 변환
회전 곱셈 순서	`R(축) * q` (월드 축 기준)	`q * R(축)` (물체 축 기준)
초기화 단계	셰이더 → 텍스처 → 기하 데이터	셰이더 → 텍스처 → 기하 데이터 → **장면**
처음 보이는 면	+z 면 (face 0)	+x 면 (face 1), y축 기준 −90° 회전 때문
카메라를 옮겼을 때	회전축이 마우스 방향과 어긋남 (회전한 카메라는 표현도 불가)	계속 마우스 방향대로 회전

5.3 현재 설정에서 두 계산 결과가 같은 이유

물체 회전을 R_o, 월드 좌표계 회전축을 n_w, 회전각을 θ라고 하면 v2의 갱신식은 다음과 같습니다.

```
R_o' = R_o · R(R_o^-1 n_w, θ)
회전의 켤레 관계 R(Q n, θ) = Q · R(n, θ) · Q^-1 에 Q = R_o^-1을 넣으면
R(R_o^-1 n_w, θ) = R_o^-1 · R(n_w, θ) · R_o
∴ R_o' = R(n_w, θ) · R_o   ← 월드 축 기준으로 앞에 곱한 것과 같음
```

지금은 카메라가 회전하지 않아서 $n_w = n_{view}$입니다. 따라서 v2의 결과는 v1의 `R(n_view, θ) · rotation`과 같습니다. v2가 실제로 더한 것은 **화면 → 월드 변환(C)** 하나이고, 이 변환 덕분에 카메라가 회전해 있어도 계산이 맞습니다.

6. 자주 하는 오해: "v2는 카메라가 움직이는 코드인가?"

아닙니다. v2의 `cameraPosition`, `cameraTarget`, `cameraUp`은 `initScene()`(cube-v2/mainwidget.cpp:110-112)에서 한 번 정해진 뒤로 바뀌지 않습니다. `mouseMoveEvent`가 바꾸는 값은 `objectOrientation` 하나뿐입니다.

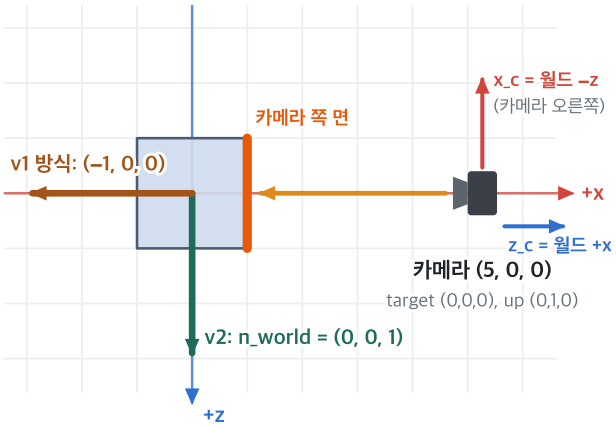
그런데도 헷갈리는 이유는 **상대 운동** 때문입니다. 화면에 물체가 하나뿐이고 배경이 검은색이면, 물체를 돌리는 것과 카메라가 물체 주위를 도는 것은 화면상 구분할 수 없습니다. 구분하려면 바닥 격자나 좌표축처럼 고정된 기준 물체가 하나 더 있어야 합니다.

7. 확인 실험: 카메라 위치 바꾸기

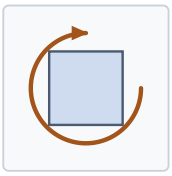
cube-v2/mainwidget.cpp:110

```
cameraPosition = QVector3D(3.0f, 3.0f, 3.0f);   // 대각선 위에서 원점을 내려다봄
```

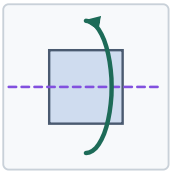
- 카메라와 원점 사이 거리는 √27 ≈ 5.2입니다. 큐브 모서리(원점에서 최대 √3 ≈ 1.73)까지 포함해도 약 3.5~6.9로, 투영의 near 3 ~ far 7 범위 안에 있습니다. 이 범위를 벗어나면 큐브가 잘려 보입니다.
- v2 이렇게 바꿔도 마우스를 끄는 방향대로 큐브가 돕니다.
- v1 카메라를 `translate`로만 표현하기 때문에 원점을 바라보는 대각선 카메라를 만들 수 없습니다. 회전을 억지로 넣으면 회전축이 마우스 방향과 어긋납니다.



위로 끌었을 때, 카메라에 보이는 모습



v1 방식
축이 시선과 평행
→ 제자리에서 회전(roll)
끈 방향과 다르게 돈다



v2
축 = 화면의 가로축
→ 앞면이 위로 기울어짐
끈 방향대로 돈다

위에서 내려다본 그림 (y축은 종이 밖). 큐브와 카메라 사이 거리 5는 v2 기본값과 같다

그림 6. 카메라를 `(5,0,0)`으로 옮기고 위로 끌었을 때입니다. 카메라의 오른쪽은 월드 −z이므로 `n_view = (−1,0,0)`은 월드에서 `(0,0,1)`입니다. v2는 이 축으로 돌려서 앞면이 위로 기울어집니다. 반면 v1처럼 `(−1,0,0)`을 그대로 월드 축으로 쓰면, 그 축이 시선과 평행해서 큐브가 화면에서 제자리로 돌기만 합니다.

8. 코드를 고칠 때 참고할 점

1. **v2의 입력 처리가 직전 프레임 행렬을 씁니다.** `modelmatrix`는 `paintGL`에서만 갱신되는데 `objectOrientation`은 마우스 이벤트마다 바뀝니다. 다시 그리기 전에 마우스 이벤트가 여러 번 오면 회전축이 아주 조금 어긋날 수 있습니다. 정확하게 하려면 최신 방향으로 직접 변환합니다.

   ```
   QVector3D n_object = objectOrientation.inverted().rotatedVector(n_world);
   ```

2. **주석이 코드와 맞지 않습니다** (cube-v1, v2 모두 geometryengine.cpp:83-89). "triangle strip" 설명이 남아 있지만 실제로는 `GL_TRIANGLES`를 씁니다. v0에서 넘어올 때 고치지 않은 주석입니다.
3. **VAO를 제대로 활용하지 못하고 있습니다.** VAO는 정점 속성 설정을 한 번 저장해 두는 용도인데, 지금은 `drawCubeGeometry()`에서 매 프레임 다시 설정합니다. 동작에는 문제가 없지만, 물체가 많아지면 초기화할 때 한 번만 설정하는 편이 맞습니다.
4. **창 크기를 전역 변수에 저장합니다.** `g_windowWidth`, `g_windowHeight`는 파일 전역 `static` 변수입니다. 위젯이 하나일 때는 문제없지만, 위젯을 여러 개 만들면 멤버 변수로 옮겨야 합니다.
5. **기능을 추가할 때는 v2에서 시작하는 것이 좋습니다.** 키보드로 카메라 이동, 물체 추가, 오빗 카메라 같은 기능은 v2 구조에서 **장면 상태 변수만 바꾸면** `paintGL`이 알아서 반영합니다.